Session 2
Review & Documentation

M2104 Object-Oriented Design and Modelling - Explaining a Program

2021 Edition

S1Ex5 in Review

Class diagram review

- **Object**: entity (individual) providing one (some) service(s) through **operations** (methods) and containing **data** (attributes)

- **Classes**: define objects (and add class methods/attributes), related by **association** (references) or **use links** (usage)

- The **UML class diagram**: classes are each in a rectangle box with their name, data, operations; visibilities are noted by symbols (+, -…), associations by **continuous lines** with **roles** and **multiplicities** (related to the nearest class), use links by arrow-ended **dashed lines** (and ‹‹ uses ››)

- Plural muliplicities are **containers** (arrays, lists, sets, associative mappings) in Java code, **association classes** represent using an object some characteristics of a pair of associated objects (sometimes using an associative mapping)

Addition: navigability

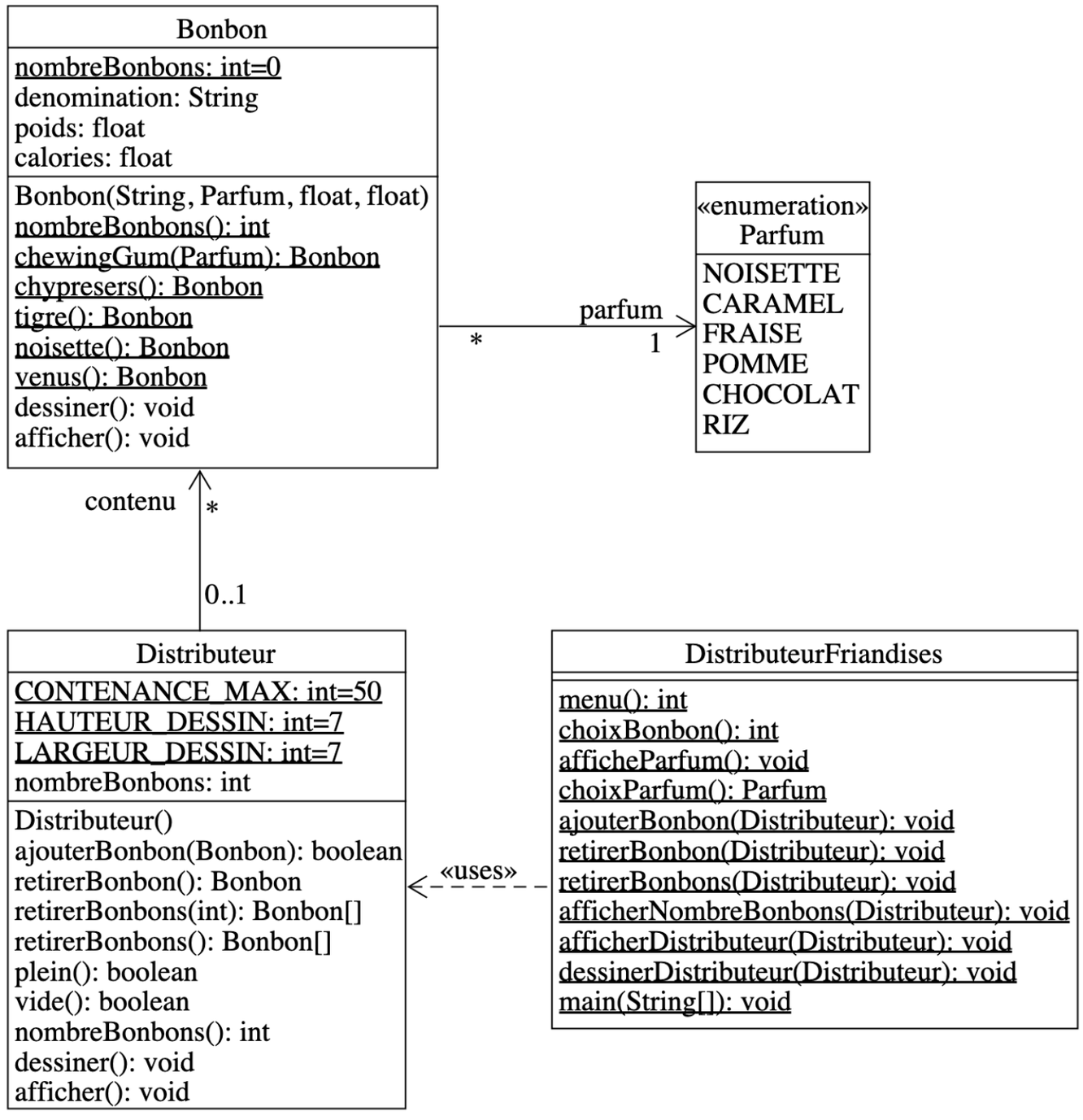
- An **arrow** (a classic arrow →) on associations (or use links) denotes the **navigability**: the direction of information

- **One-way navigability on an association**: a single arrow, one of the associated classes refers to the other, not the other way around

- **Two-way navigability**: two arrows (one at each end), each class refers to the other

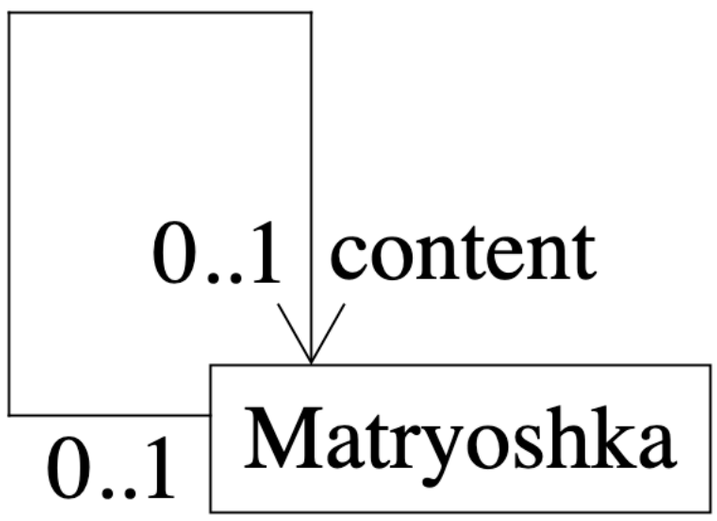
- **Underspecified (absent) navigability**: no arrow (either two-way or not important for this diagram)

Example : *Candy vending machine* with navigability



Addition: reflexive associations

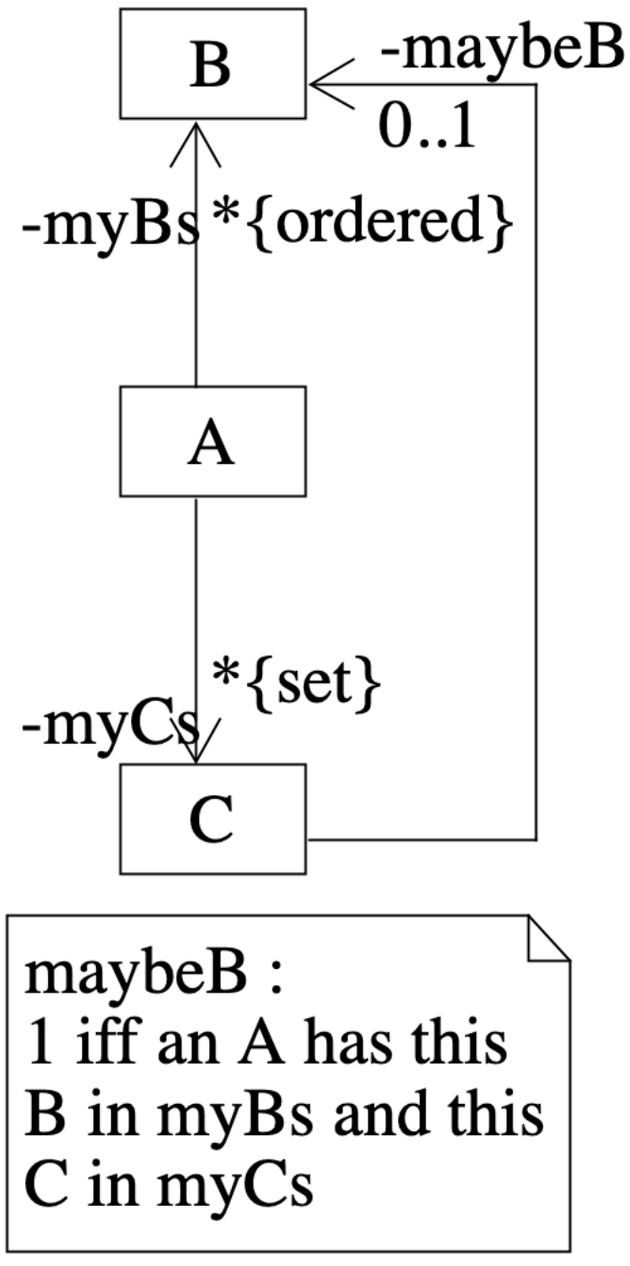
- A class may be associated to itself when objects contain references to their own type (parent, successor…)
- "Russian dolls":



Addition: other things

- Many things are **optional** in the class diagram (visibility, types, &c.)
- We may add (seldom) :
  - A characterisation on the multiplicities of an association such as `set` or `ordered` (specific collection)
  - Visibility on association roles
  - Plain text notes…

Example : decorated diagram



Exercise 2 : Java code transcription

```
/**
 * A simple social network
 */
public class Network {
    private Set<Member> accounts;
    /**
     * Access member accounts by name.
     * @param memberName the name of the account to search for
     * @return member account or null if absent
     */
    public Member getAccountOrNull(String memberName) {}
}
/**
 * The account of a member
 */
public class Member {
    private String name;
    private Map<Member, Relationship> friends;
    /**
     * Tests if this member has another member in their "friends"
list
     * @param other the different member to check
     * @return true iff the two are connected as "friends".
     */
    public boolean knows(Member other) {}
    /**
     * Adds the other member as a friend, and reciprocate.
     * @param other the new friend
     */
    public void addToFriends(Member other) {}
}
```

```
/**
 * Class describing the relationship between two members.
 */
public class Relationship {
    private Member a, b;
    private Date initiatedOn;
    private String description="my new friend";
    public Relationship(Member a, Member b) {}
    /**
     * Creates a text representation of the relationship.
     * @return the String representation of the object
     */
    @Override
    public String toString() {}
    /**
     * Changes the description of the relationship.
     * @param desc the new description to use (String).
     */
    public void setDescription(String desc) {}
}
```

Lesson: the importance of code documentation

- To document code is to **write documentation comments** (no substitute for the user manual; does not consist in putting comments in the code)

- **Essential** in projects with several people/teams: even if the code is perfectly written, it **should never be necessary to read code** in order to understand how to use it (in other classes)

- **Necessary** in any project (even in one-person teams) to make sense of your own code…

Javadoc

- In Java : Javadoc (other languages/communities have other norms)
- **Public methods**:
  - `@param name description`: for all **parameters**
  - `@return description`: for the **return value** (if it is not an action)
  - `@throws type description`: for all (explicitly) thrown **exceptions**
- **Classes**:
  - `@author name`: for each author (developper, sometimes left out in this course)
  - `@version number`: class version
- **Also**: `@see other`, `@since version` (whenever needed)

Exercise 4 : Documenting existing code

```
/**
 * A class representing data about a building or house.
 * @author Bruno T. Mery
 * @version 0.0
 */
public class Building {
    private String address;
    private String type;
    private Map<Pe, Pl> tenants;
    /**
     * Parameterised constructor creating an empty building.
     * @param address the post address
     * @param type a text such as "tower", "house", "dormitory"…
     */
    public Building(String address, String type) {
        this.address = address;
        this.type = type;
        tenants = new HashMap<>();
    }
    /**
     * A method that adds a new tenant in this building.
     * @param tenant the new person in the building
     * @param room the apartment/room of the new tenant
     */
    public void addTenant(Pe tenant, Pl room) {
        tenants.put(tenant, room);
        tenant.i(this, room);
    }
}
/**
 *
 *
 */
public class Pe {
    private String n;
    private Building b;
    private Pl r;
    /**
     *
     *
     */
    public Pe(String n) {
        this.n=n;
    }
    /**
     *
     *
     */
    void i(Building b, Pl r) {
        this.b=b;
        this.r=r;
        this.r.i(this);
    }
}
/**
 *
 *
 */
public class Pl {
    private int n;
    private String t;
    private Building b;
    private Set<Pe> p;
    /**
     *
     *
     */
    public Pl(String t, Building b, int n) {
        this.n=n;
        this.b=b;
        this.t=t;
        this.p = new HashSet<>();
    }
    /**
     *
     *
     */
    void i(Pe p) {
        this.p.add(p);
    }
}
```

Time allowing
Bonus questions…

- List all the *names* given (variables, attributes…), give more fitting names

- Is `Pl` an association class? why?

- Give the class diagram for this code

- (Training…)
  Do the class diagrams for the previous classes/practicals in M2103